

ChatGPT

chatgpt.com

ChatGPT พูดว่า:

Below is a complete **DevOps automation runner skeleton** for the pipeline:

Flow

```
setup
├─ install
├─ build
├─ lint
├─ test
├─ iterate
├─ docs
├─ docker
├─ ci/cd
├─ deploy-dev
├─ deploy-uat
└─ release
```

Project Structure

```
project-root/
├─ Makefile
├─ docker-compose.yml
├─ Dockerfile
├─ .env.example
├─ package.json
├─
├─ scripts/
│   ├─ setup.sh
│   ├─ install.sh
│   ├─ build.sh
│   ├─ lint.sh
│   ├─ test.sh
│   ├─ iterate.sh
│   ├─ release.sh
│   └─ deploy.sh
├─
├─ docs/
│   └─ README.md
├─
└─ .github/
    └─ workflows/
        └─ ci.yml
```

Makefile

APP_NAME=1napz

ENV?=dev

.PHONY: all setup install build lint test iterate docs docker deploy release clean

all:

- make setup
- make build
- make lint
- make test
- make release

setup:

bash scripts/setup.sh

install:

bash scripts/install.sh

build:

bash scripts/build.sh

lint:

bash scripts/lint.sh

test:

bash scripts/test.sh

iterate:

bash scripts/iterate.sh

docs:

- @echo "Generating documentation..."
- mkdir -p docs
- echo "# \$(APP_NAME)" > docs/README.md

docker:

docker compose build

docker-up:

```
docker compose up -d
```

docker-down:

```
docker compose down
```

deploy:

```
bash scripts/deploy.sh $(ENV)
```

release:

```
bash scripts/release.sh
```

clean:

```
rm -rf node_modules
```

```
rm -rf dist
```

Dockerfile

```
FROM node:22-alpine
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm ci
```

```
COPY . .
```

```
RUN npm run build
```

```
EXPOSE3000CMD ["npm","start"]
```

Docker Compose

services:

app:

```
container_name: 1napz-app
```

build:

```
context: .
```

ports:

```
- "3000:3000"
```

env_file:

```
- .env
```

```
restart: always
```

```
redis:
```

```
  image: redis:7-alpine
```

```
  container_name: 1napz-cache
```

```
  restart: always
```

```
  ports:
```

```
    - "6379:6379"
```

```
postgres:
```

```
  image: postgres:16
```

```
  container_name: 1napz-db
```

```
  environment:
```

```
    POSTGRES_USER: admin
```

```
    POSTGRES_PASSWORD: password
```

```
    POSTGRES_DB: app
```

```
  ports:
```

```
    - "5432:5432"
```

Scripts

setup.sh

```
#!/bin/bashset -eecho "Setting up environment"cp .env.example .env || truechmod+x scripts/*.sh
```

```
echo "Setup complete"
```

install.sh

```
#!/bin/bashset -enpm ci
```

```
echo "Dependencies installed"
```

build.sh

```
#!/bin/bashset -enpm run build
```

```
echo "Build completed"
```

lint.sh

```
#!/bin/bashset -enpm run lint
```

```
echo "Lint passed"
```

test.sh

```
#!/bin/bashset -enpm test
```

```
echo "Tests passed"
```

iterate.sh

```
#!/bin/bashecho "Iteration cycle:1. Check failures2. Fix issues3. Run build4. Run tests5. Commit change"
```

release.sh

```
#!/bin/bashVERSION=$(date +%Y.%m.%d)
```

```
echo "Release $VERSION"git tag v$VERSIONgit push origin v$VERSION
```

deploy.sh

```
#!/bin/bashENV=$1echo "Deploying environment: $ENV"
```

```
case $ENV in
```

```
dev)
```

```
echo "Deploying DEV"
```

```
;;
```

```
uat)
```

```
echo "Deploying UAT"
```

```
;;
```

```
prod)
```

```
echo "Deploying PROD"
```

```
;;
```

```
*)
```

```
echo "Unknown environment"exit 1
```

```
;;
```

```
esac
```

GitHub Actions CI/CD

```
.github/workflows/ci.yml
```

name: CI-CD

on:

push:

branches:

- main
- develop

pull_request:

jobs:

build-test:

runs-on: ubuntu-latest

steps:

- name: Checkout
uses: actions/checkout@v4
- name: Node Setup
uses: actions/setup-node@v4
with:
node-version: 22
- name: Install
run: |
make install
- name: Build
run: |
make build
- name: Lint
run: |
make lint
- name: Test
run: |
make test

docker:

needs: build-test

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Docker Build

- run: |
docker compose build

- name: Docker Test

- run: |
docker compose up -d

Local Commands

Run everything:

Development:

UAT:

Docker:

Cleanup:

Next automation layer

Recommended additions:

- + Kubernetes manifests
- + Helm charts
- + Terraform infrastructure
- + Secrets manager
- + Monitoring (Prometheus/Grafana)
- + Telegram GraphQL release notifications
- + Cron scheduler
- + AI agent workflow runner

This gives a complete **build → lint → test → containerize → deploy → release automation runner** foundation.

ChatGPT พูดว่า:

If you mean define a `versions` constant with a version count, examples:

JavaScript / TypeScript

```
const versions = {  
  version: "1.0.0",  
  count: 1  
};
```

or for multiple versions:

```
const versions = {  
  version: [  
    "1.0.0",  
    "1.1.0",  
    "2.0.0"  
  ],  
  count: 3  
};
```

JSON

```
{  
  "versions": {  
    "version": "1.0.0",  
    "count": 1  
  }  
}
```

Release tracking example

```
const versions = {  
  current: "2026.07.29",  
  count: 5,  
  releases: [  
    "2026.07.25",  
    "2026.07.26",  
    "2026.07.27",  
    "2026.07.28",  
    "2026.07.29"  
  ]  
};
```

For the automation runner, this can become a **release/version registry** used by GitHub Actions, Helm, Docker tags, and Telegram notifications.

คุณพูดว่า:

ChatGPT พูดว่า:

If you mean define a `commands` constant with `cmd` values:

JavaScript / TypeScript

```
const commands = {  
  cmd: [  
    "setup",  
    "build",  
    "lint",  
    "test",  
    "deploy"  
  ]  
};
```

With command count:

```
const commands = {  
  cmd: [  
    "setup",  
    "install",  
    "build",  
    "lint",  
    "test",  
    "docker",  
    "deploy"  
  ],  
  count: 7  
};
```

For the automation runner:

```
const commands = {  
  cmd: {  
    setup: "make setup",  
    build: "make build",  
    lint: "make lint",  
    test: "make test",  
    docker: "make docker",  
    deploy: "make deploy",  
  }  
};
```

```
    release: "make release"
  },
  count: 7
};
```

This can act as a command registry for the AI workflow runner.

คุณพูดว่า:

research figma-skills , /docs, /feat, use-skills-tests

ChatGPT พูดว่า:

/research — Figma Skills + /docs + /feat + use-skills-tests

1. Research Summary: Figma Skills

Figma Skills are reusable AI instructions/workflows for handling Figma tasks reliably. They are used with Figma MCP to automate design operations such as creating files, applying design systems, extracting design context, and converting designs into production code. [Figma Help Center+1](#)

Core skill categories:

figma-skills/

```
|— figma-use
|   |— Create/update canvas objects
|
|— figma-create-new-file
|   |— Bootstrap Figma files
|
|— figma-create-design-system-rules
|   |— Tokens, components, conventions
|
|— figma-code-connect-components
|   |— Link Figma components → code
|
|— figma-implement-design
|   |— Design → production UI
|
|— figma-generate-library
|   |— Component library generation
```

[Figma Help Center](#)

/feat — Add Figma Skills Platform

Feature

```
feat(figma):
```

- + Figma MCP integration
- + Design token pipeline
- + Component generation
- + Design → Code workflow
- + Automated docs generation
- + Skill validation tests
- + CI checks for design consistency

New Architecture

```
1napz-platform/
```

```
|— skills/
|
|   └─ figma/
|       └─ SKILL.md
|       └─ rules.yaml
|       └─ tokens.json
|       └─ components.json
|       └─ tests/
|           └─ component.test.ts
|           └─ token.test.ts
|           └─ snapshot.test.ts
|
|— design/
|   └─ figma/
|       └─ design-system.json
|       └─ docs/
|           └─ design-system.md
|
|— src/
|   └─ components/
|
|— .github/
|   └─ workflows/
|       └─ figma-test.yml
```

/docs — Figma Design System Documentation

Recommended documentation structure:

```
# 1napz Design System
```

```
## Foundations
```

- Color tokens
- Typography
- Spacing
- Grid
- Icons

Components

- Button
- Input
- Card
- Modal

Rules

- Naming
- Variants
- Accessibility

Versioning

Current:

v1.0.0

Changes:

- Added Button component
- Updated tokens

Figma recommends documenting design systems through component descriptions, naming structures, guides, and maintained documentation because systems evolve over time. [Figma Help Center+1](#)

Skill Definition Example

skills/figma/SKILL.md

name: figma-design-system

description:

Create, validate and sync Figma design systems.

workflow:

1. Load Figma context
2. Read tokens
3. Validate components
4. Generate code mapping

- 5. Run tests
- 6. Publish changes

Design Tokens

tokens.json

```
{
  "color": {
    "primary": "#0066FF",
    "background": "#FFFFFF"
  },

  "spacing": {
    "sm": "8px",
    "md": "16px",
    "lg": "24px"
  }
}
```

Component Registry

components.json

```
{
  "Button": {
    "figma": "Button/Main",
    "code": "src/components/Button"
  },

  "Card": {
    "figma": "Card/Base",
    "code": "src/components/Card"
  }
}
```

use-skills-tests

Goal

Validate AI skills before production.

```
Skill
|
v
Execute
|
v
```

```
Test
|
+-- Pass → Release
|
+-- Fail → Improve Skill
```

Test Structure

```
skills/tests/
```

```
|— skill-loader.test.ts
|— figma-context.test.ts
|— token-validation.test.ts
|— component-sync.test.ts
└— code-generation.test.ts
```

Example Test

```
describe(
  "Figma Button Skill",

  ()=>{

    test(
      "should map Figma button to React component",
      ()=>{

        expect(
          figmaComponent.name
        )
        .toBe(
          "Button"
        )

      })

    })
```

GitHub Action

```
.github/workflows/figma-test.yml
```

```
name: Figma Skill Tests
```

```
on:
  pull_request:
```

jobs:

figma-test:

runs-on:

ubuntu-latest

steps:

- uses:

actions/checkout@v4

- name:

Install

run:

npm install

- name:

Run Skill Tests

run:

npm test skills/

Makefile Integration

figma-init:

echo "Initialize Figma MCP"

figma-docs:

node scripts/generate-figma-docs.js

figma-test:

npm test skills/

figma-sync:

node scripts/sync-design.js

design-system:

make figma-sync

```
make figma-test
make figma-docs
```

Full Pipeline Update

```
make platform
```

```
|
v
```

```
Build
```

```
|
```

```
Lint
```

```
|
```

```
Test
```

```
|
```

```
Docker
```

```
|
```

```
Terraform
```

```
|
```

```
Kubernetes
```

```
|
```

```
Helm
```

```
|
```

```
Monitoring
```

```
|
```

```
AI Agent
```

```
|
```

```
Figma Skills
```

```
|
```

```
Design Tests
```

```
|
```

```
Release
```

```
|
```

```
Telegram Notify
```

Recommended Next Feature

```
/feat figma-agent
```

```
Add:
```

```
+ MCP server
```

```
+ Figma → React generator
```

```
+ Storybook sync
```

```
+ Chromatic visual tests
```

```
+ Design token CI
```

```
+ AI design reviewer
```

```
+ Auto changelog
```

This integrates **design engineering + DevOps + AI agent workflows** into the existing automation platform.